Automata and Complexity Theory CoSc4311_ CoSc4312_

June 15, 2025

Compiled by: Natan Asrat

https://www.linkedin.com/in/natan_asrat

Automata and Complexity Theory

- **Theory of Automata**
  - theoretical branch of computer science and mathematics
  - study of abstract machines and the computation problems that can be solved using these machines
  - the abstract machine is called the automata
  - automaton consists of states and transitions.
    - State is represented by circles, and
    - Transition is represented by arrows

Automata and Complexity Theory

- **Theory of Automata**
  - Automata is the kind of machine that takes some string as input and this input goes through a finite number of states and may enter the final state
    - Symbols: an entity or individual objects, which can be any letter, alphabet or any picture
    - Alphabet: finite set of non-empty symbols. It is denoted by ∑ (sigma)
    - String: finite collection of symbols from the alphabet. The string is denoted by w.

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Language: collection of appropriate string.
      - can be Finite or Infinite.
    - Grammar: defines a set of rules, and with the use of these rules, valid sentences in a language are generated
      - popular way to derive a grammar recursively is phrase-structure grammar
    - Finite Automata:
      - used to recognize patterns
      - takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs
      - at the time of transition, the automata can either move to the next state or stay in the same state
      - have two states, Accept state or Reject state

Automata and Complexity Theory

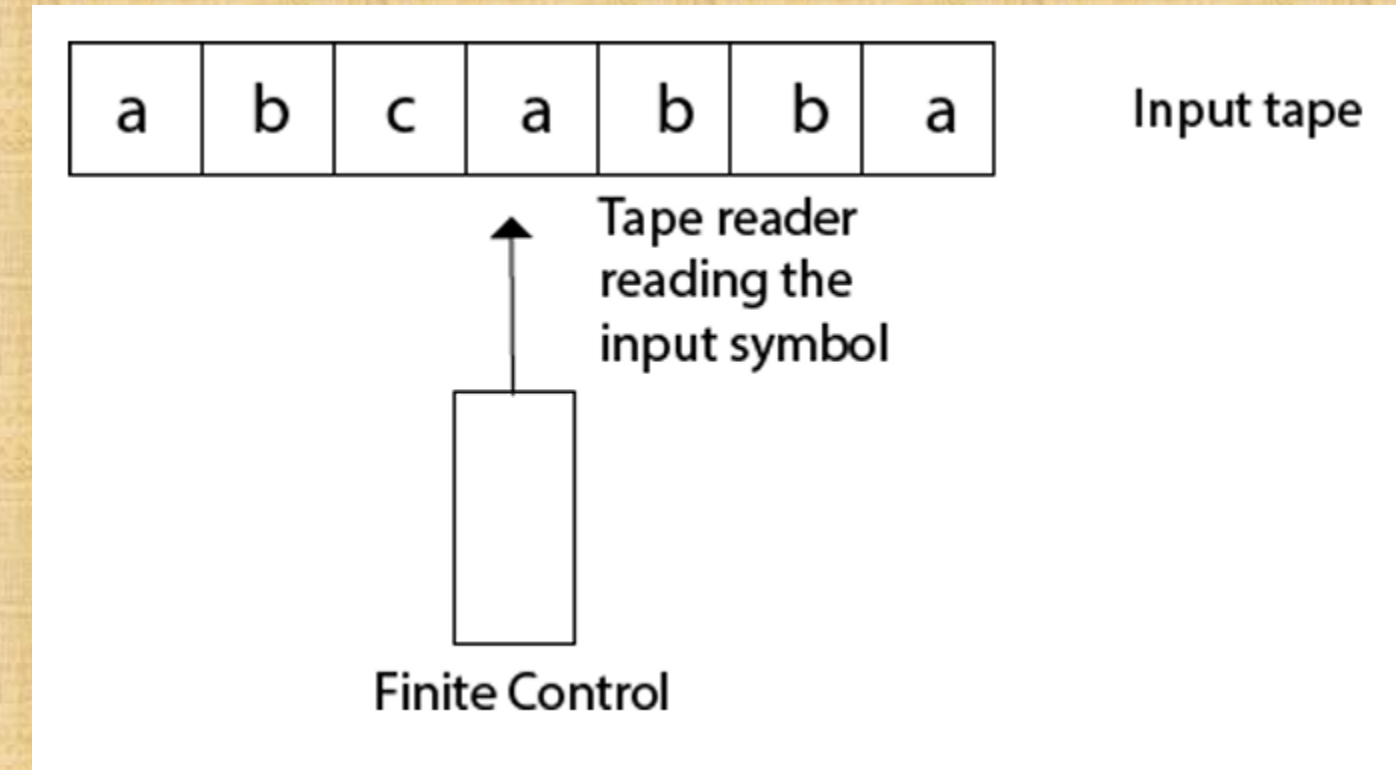
- **Theory of Automata**
  - Automata:
    - Language: collection of appropriate string.
      - can be Finite or Infinite.
    - Grammar: defines a set of rules, and with the use of these rules, valid sentences in a language are generated
      - popular way to derive a grammar recursively is phrase-structure grammar
    - Finite Automata:
      - used to recognize patterns
      - takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs
      - at the time of transition, the automata can either move to the next state or stay in the same state
      - have two states, Accept state or Reject state

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Formal Definition of FA
      - collection of 5-tuple (Q, ∑, δ, q0, F)
        - Q: finite set of states
        - ∑: finite set of the input symbol
        - q0: initial state
        - F: final state
        - δ: Transition function

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Finite Automata Model: represented by input tape and finite control



Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Types of Automata:
      - DFA(deterministic finite automata)
        - machine goes to one state only for a particular input character.
        - does not accept the null move
      - NFA(non-deterministic finite automata)
        - transmit any number of states for a particular input
        - can accept the null move
      - Every DFA is NFA, but NFA is not DFA
      - There can be multiple final states in both NFA and DFA
      - DFA is used in Lexical Analysis in Compiler
      - NFA is more of a theoretical concept

Automata and Complexity Theory

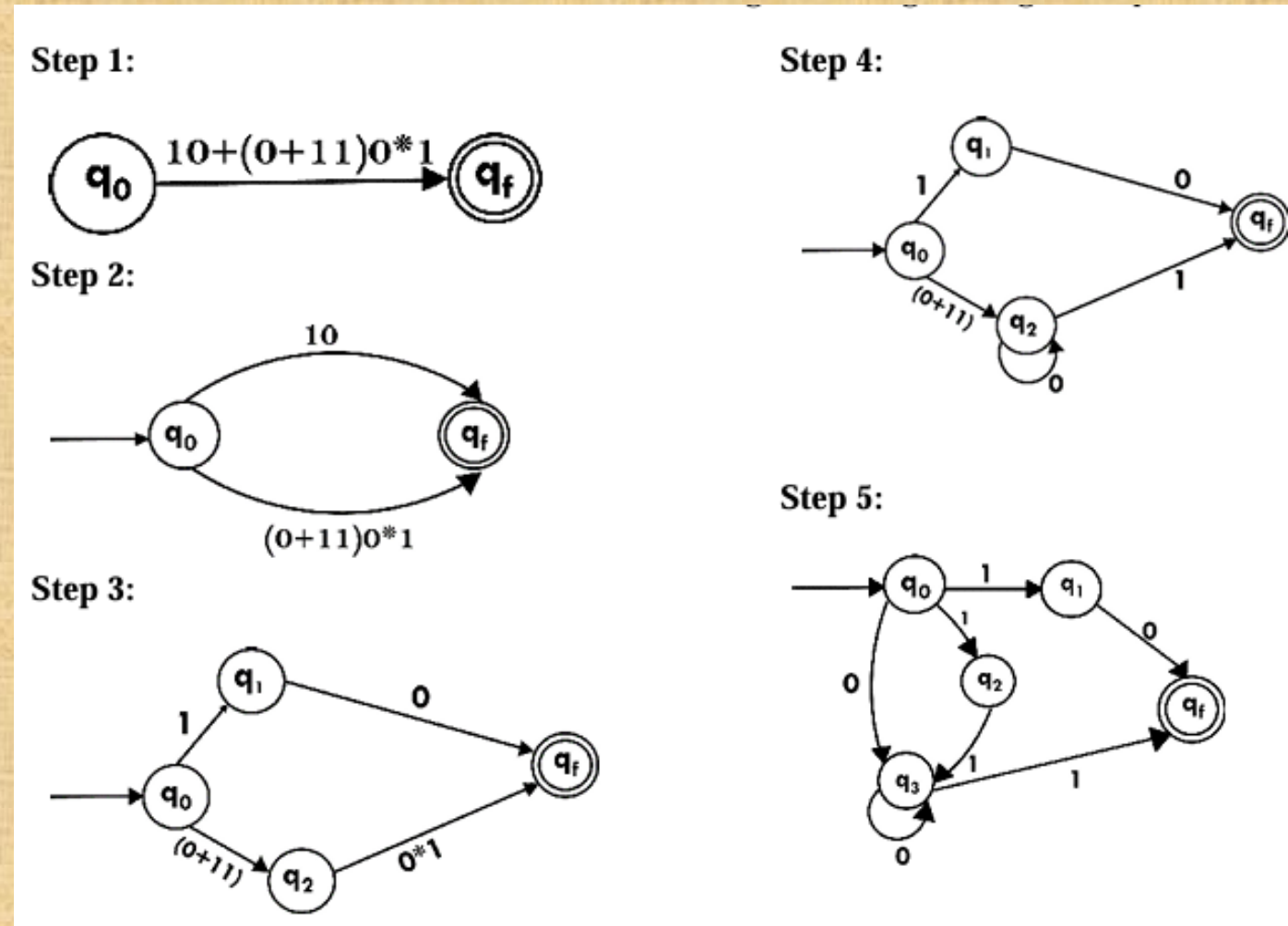
- **Theory of Automata**
  - Automata:
    - Regular Expression
      - description of language accepted by finite automata
      - languages accepted by some regular expression are referred to as Regular languages
      - x* means zero or more occurrence of x.
      - x+ means one or more occurrence of x
    - Operations on Regular Language
      - Union
      - Intersection
      - Kleen closure: L* = Zero or more occurrence of language L

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Conversion of RE to FA
      1. Design a transition diagram for given regular expression, using NFA with ε moves.
      2. Convert this NFA with ε to NFA without ε.
      3. Convert the obtained NFA to equivalent DFA.

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Conversion of RE to FA



Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Regular Languages
      - language that can be described with regular expressions
      - Regular Languages (LREG) is the set of all regular languages
      - language is regular language if and only if some finite state machine recognizes it
      - languages accepted by all DFAs form the family of regular languages

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Regular Languages
      - Pumping Lemma:
        - For any regular language L, there exists an integer n, such that
          - for all $x \in L$ with $|x| \geq n$, there exists $u, v, w \in \Sigma^*$,
          - such that $x = uvw$, and
            - (1) $|uv| \leq n$
            - (2) $|v| \geq 1$
            - (3) for all $i \geq 0$: $uviw \in L$
        - this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L
        - if a language is regular, it always satisfies pumping lemma

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Context free languages
      - Context-free grammar: set of recursive rules used to generate patterns of strings
        - can be modelled as parse trees
          - nodes of the tree represent the symbols and the edges represent the use of production rules
          - leaves of the tree are the end result (terminal symbols) that make up the string the grammar is generating
        - described by a four-element tuple (V, Σ, R, S)
          - V is a finite set of variables (which are non-terminal)
          - Σ is a finite set (disjoint from V) of terminal symbols
          - R is a set of production rules
          - S (which is in V) which is a start symbol

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Context free languages
      - Context-free grammar:
        - sentential form: any string derivable from the start symbol
          - sentence is a sentential form consisting only of terminals
        - derivation tree: graphical representation of production rules in a CFG
        - Simplification of grammar: reduction of grammar by removing useless symbols
          - Reduction of CFG,
          - Removal of Unit Productions,
          - Removal of Null Productions

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Context free languages
      - Pushdown automata
        - nondeterministic finite state machines augmented with additional memory (stack)
        - computational models (theoretical computer-like machines)
          - can do more than a finite state machine,
          - but less than a Turing machine
        - formally defined as a 7-tuple: (Q, Σ, Γ, δ, q0, Z, F)
        - useful when thinking about parser design and any area where context-free grammars are used
      - there are two ways of proving that a language is context-free
        - provide the context-free grammar
        - provide a pushdown automaton for the language

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines:
      - first described by Alan Turing,
      - simple abstract computational devices intended to help investigate the extent and limitations of what can be computed
      - a model of general purpose computer
      - can do everything that a real computer can do
      - uses an infinite tape as its unlimited memory
      - has a tape head that can read and write symbols and move around on the tape

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines:
      - defined as a 7-tuple, namely (Q, ∑, ┌, δ, q0, b, F)
        - Q is a finite nonempty set of states
        - ∑ is a nonempty set of input symbols and is subset of ┌ and b∉∑
        - ┌ is a finite nonempty set of tape symbols
        - b∉┌ is the blank symbol
        - δ is the transition function, mapping (q, x) onto (q', y, D) where
          - D denotes the direction of movement of R/W head:
            - D = L or R (left or right)
          - δ may not be defined for some elements of QX┌
        - q0∈ Q is the initial state , and
        - F⊆Q is the set of final states

Automata and Complexity Theory

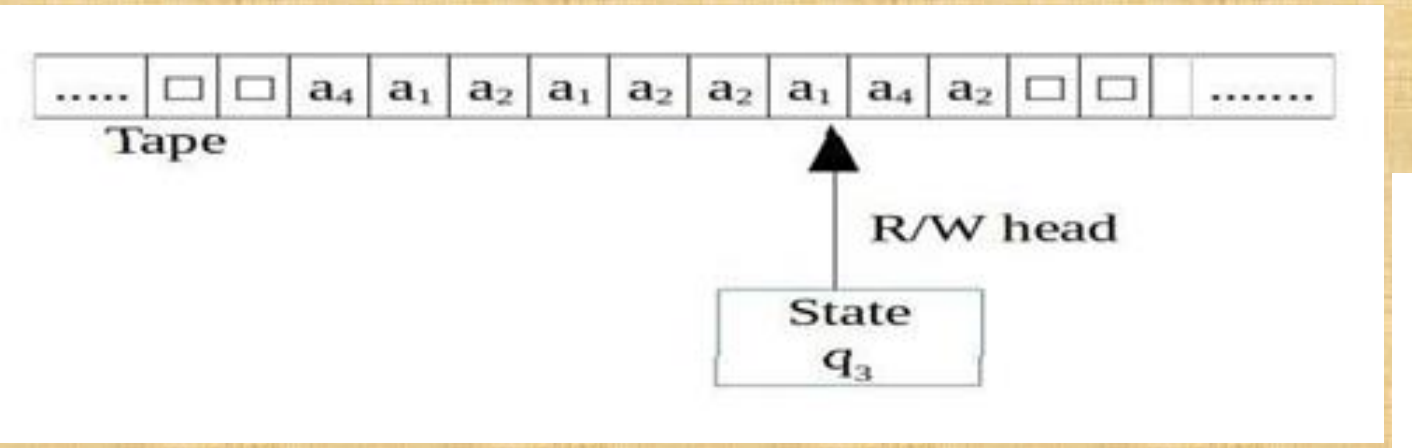
- **Theory of Automata**
  - Automata:
    - Turing machines:
      - TMs are deterministic: for each state there is only one unique Transition on each symbol
      - Partial Transition Function: no transition for all input symbol

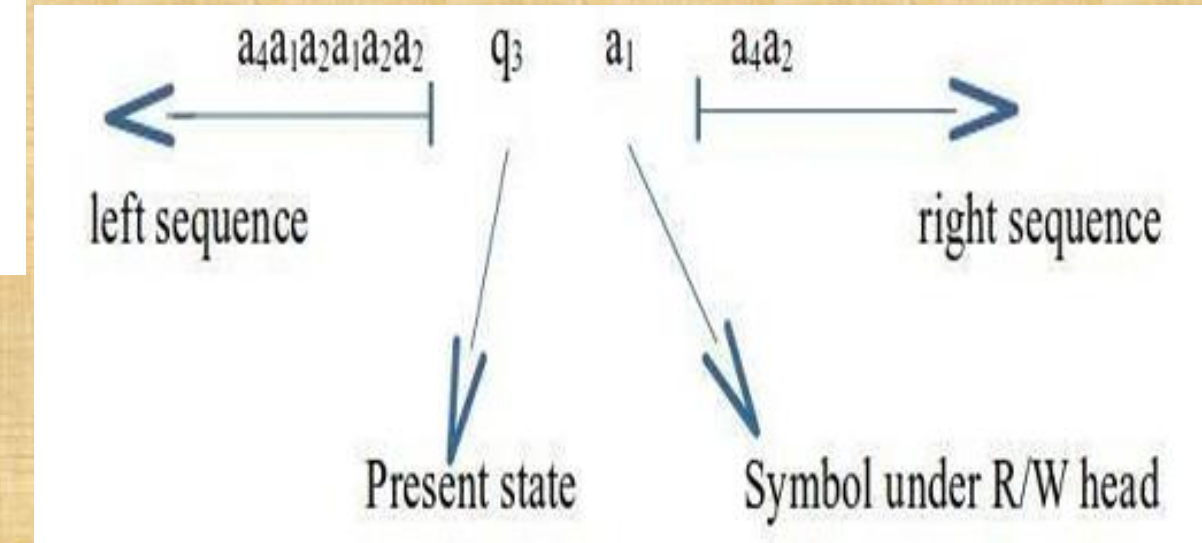
Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Instantaneous descriptions
        - defined in terms of the entire input string and the current state
        - written as $\alpha\beta\gamma$, where
          - β is the present state of M, and
          - input string is split as $\alpha\gamma$, the first symbol of γ is the current symbol under the R/W head

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Instantaneous descriptions





Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Instantaneous descriptions
        - defined in terms of the entire input string and the current state
        - written as $\alpha\beta\gamma$, where
          - β is the present state of M, and
          - input string is split as $\alpha\gamma$, the first symbol of γ is the current symbol under the R/W head

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Transition table
        - definition of transition function δ in the form of a table
        - If δ(q, a) = (γ, α, β), we write αβγ under the α-column and in the q-row
          - αβγ
            - α is written in the current cell
            - β gives the movement of the head (L or R)
            - γ denotes the new state
        - initial state is marked with → and the final state is marked with a circle

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Transition table

Present State	Tape Symbol		
	□	0	1
$\rightarrow q_1$	1 L q_2	0 R q_1	-
q_2	□ R q_3	0 L q_2	1 L q_2
q_3	-	□ R q_4	□ R q_5
q_4	0 R q_5	0 R q_4	1 R q_4
q_5	0 L q_2		

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines - Representation of Turing Machine
      - Transition diagram (transition graph)
        - states are represented by vertices
        - Directed edges are used to represent transition of states.
          - for directed edge from qi to qj with label (α,β,γ), it means
            - δ(qi, α) = (qj, β, γ)

Automata and Complexity Theory

- **Theory of Automata**
  - Automata:
    - Turing machines:
      - possible outcomes of executing a Turing machine over a given input
        - Halt and accept the input
        - Halt and reject the input
        - Never halt (infinite loop)
      - Decidable language (recursive language): there is a TM (decider) which accepts and halts on every input
      - Recursive enumerable languages: TM halts sometimes & may not halt sometimes
        - not Recursively Enumerable that language is not accepted by any Turing Machine

Automata and Complexity Theory

- Computational complexity
  - Time Complexity: approximate number of operations required to solve a problem of size n
  - Space Complexity: approximate memory required to solve a problem of size n
  - Big-O notation: most commonly used notation for specifying asymptotic complexity

Automata and Complexity Theory

- Computational complexity
  - Big-O classes:
    - Constant Time Complexity O(1)
    - Logarithmic time : log n
    - Linear time O (n)
    - Quasilinear Time O(n log n)
    - Quadratic Time $O(n^2)$
    - Exponential time $O(2^n)$

Automata and Complexity Theory

- Computational complexity
  - Polynomial and Non Polynomial Class

Polynomial	Non Polynomial
P problems are set of problems which can be solved in polynomial time by deterministic algorithms.	NP problems are the problems which can be solved in non-deterministic polynomial time.
The problem belongs to class P if it‘s easy to find a solution for the problem.	The problem belongs to NP, if it‘s easy to check a solution that may have been very tedious to
find	
P Problems can be solved and verified in polynomial time. P problems are subset of NP problems.	Solution to NP problems cannot be obtained in polynomial time
P problems are subset of NP problems	NP problems are superset of P problems